

Python

MySQL и Python



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение



Введение в HTML



Парсинг веб-страниц с помощью BeautifulSoup



Обработка таблиц

План занятия

- Работа с MySQL из Python
- Подключение к базе данных
- Работа с курсором
- Получение результатов запроса
- Параметризованные запросы
- Контекстный менеджер with
- Метод commit



ОСНОВНОЙ БЛОК





Работа с MySQL из Python

Большие объёмы данных



Когда программы становятся больше, появляется необходимость хранить и обрабатывать большие объёмы данных — о товарах, пользователях, сообщениях, заказах и т.д.

Использование баз данных позволяет:

-  Хранить структурированные данные вне программы
-  Быстро искать, обновлять и удалять записи
-  Работать с очень большими объёмами данных без загрузки их всех в память
-  Обеспечить многопользовательский доступ и защиту данных
-  Делать программы более надёжными и масштабируемыми

Базы данных



Для **Python** есть удобные библиотеки для соединения и работы с базами данных.



Подключение к базе данных

Работа в Python с MySQL

Нужно использовать специальную библиотеку, которая умеет:



устанавливать соединение с сервером базы данных



отправлять SQL-запросы и получать результаты



обрабатывать ошибки при работе с базой

Популярные библиотеки для работы с MySQL

Библиотека	Особенности
PyMySQL	Простая, полностью на Python
mysql-connector-python	Официальная библиотека от MySQL, не всегда удобно интегрируется
MySQLdb (mysqlclient)	Очень быстрая, но сложнее в установке (особенно на Windows)

Установка библиотеки PyMySQL

```
pip install pymysql
```

В этой лекции мы будем использовать PyMySQL, потому что она работает стабильно на всех операционных системах

Создание соединения

Чтобы подключиться к серверу MySQL, нужно:

- Импортировать модуль `pymysql`,
- Вызвать функцию `pymysql.connect()`,
- Передать в неё параметры подключения.

Синтаксис

```
import pymysql                # импорт модуля

connection = pymysql.connect(
    host="localhost",         # адрес сервера базы данных
    user="root",              # имя пользователя
    password="yourpassword",  # пароль для указанного пользователя
    database="yourdatabase",  # название базы данных (необязательно)
    charset="utf8mb4",        # кодировка соединения (необязательно, указывать для поддержки русского)
)
```

Создание соединения

Особенности:

- Параметры подключения можно передавать **вручную**.
- Либо можно **хранить параметры в словаре** и **распаковать его** с помощью ****** при подключении.

Пример

```
import pymysql

config = {'host': 'ich-db.edu.itcareerhub.de',
          'user': 'ich1',
          'password': 'password',
          'database': 'hr',
          }

connection = pymysql.connect(**config) # распаковка словаря как аргументы
```

Когда удобно использовать словарь



Когда параметры нужно передавать в разных частях программы.



Когда подключений будет много — например, к разным базам данных.



Когда параметры удобно хранить в файлах (.env, JSON, YAML и т.д.).

Проверка соединения

```
if connection.open:  
    print("Connection successful!")
```

Заккрытие соединения

```
if connection.open:
    print("Connection successful!")
```

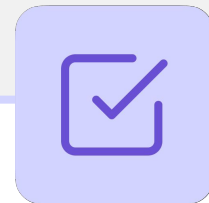
Почему важно закрывать соединение

- Открытые соединения занимают ресурсы сервера базы данных.
- Если не закрывать соединения, можно быстро исчерпать лимит подключений.
- Правильная практика: **всегда закрывать соединение**, как только оно больше не нужно.



Работа с курсором

Работа с курсором



После подключения к базе данных необходимо создать **объект курсора**, через который:

- отправляются SQL-запросы в базу,
- получаются результаты выполнения запросов.

Создание курсора



Синтаксис

```
cursor = connection.cursor()
```

Пояснения

- `cursor()` — метод соединения (`connection`), который создаёт новый курсор для работы.
- Один `cursor` используется для выполнения одного или нескольких запросов.

Заккрытие курсора



Синтаксис

```
cursor.close()
```

Пояснения

- После использования курсор нужно **закрыть вручную**, чтобы освободить ресурсы
- После закрытия курсор становится недействительным и для новых запросов нужно создать новый курсор

Выполнение запросов



Пример

```
cursor = connection.cursor()  
cursor.execute("SELECT * FROM departments")
```

Пояснения

- После создания курсора можно **выполнять SQL-запросы** с помощью метода `execute()`

Особенности работы с курсором

-  `cursor.execute(sql)` — отправляет SQL-запрос к базе данных.
-  SQL-запрос передаётся в виде строки.
-  После выполнения запроса можно получить результат.
-  Можно выполнять любой SQL-запрос: `SELECT`, `INSERT`, `UPDATE`, `DELETE` и др.
-  После каждого нового запроса старые результаты очищаются (если результат не считать, некоторые библиотеки, например `mysql-connector`, вызовут ошибку при следующем запросе).
-  При ошибке в запросе выбрасывается исключение.



ВОПРОСЫ





ЗАДАНИЯ

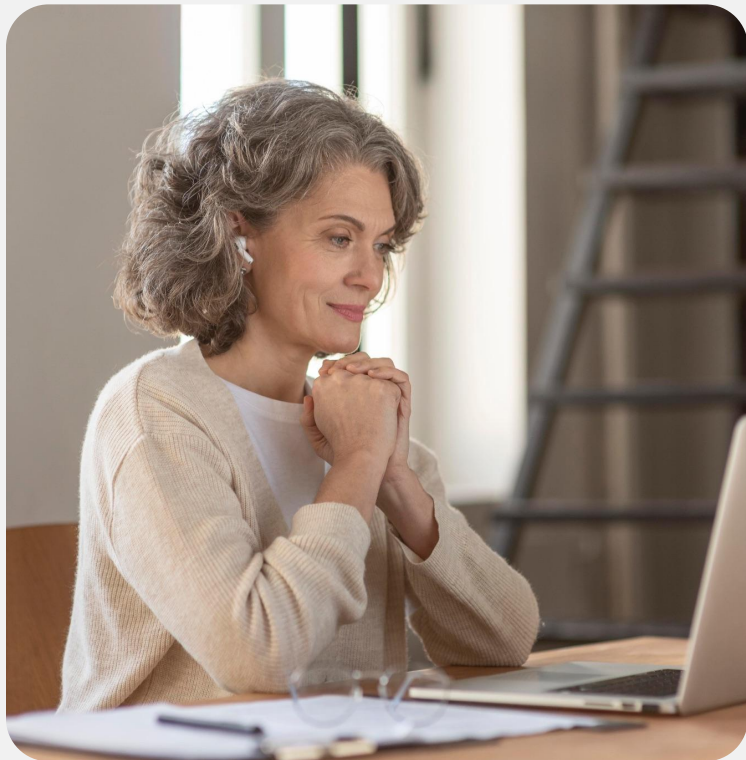




Выполните задание

Сопоставьте понятие с описанием:

1. `pymysql.connect()`
 2. `cursor.execute()`
 3. `connection.close()`
 4. `cursor = connection.cursor()`
-
- a. Закрывает соединение с базой данных
 - b. Выполняет SQL-запрос
 - c. Создает курсор для работы с базой
 - d. Создает подключение к MySQL



Выполните задание

Сопоставьте понятие с описанием:

1. `pymysql.connect()`
 2. `cursor.execute()`
 3. `connection.close()`
 4. `cursor = connection.cursor()`
-
- a. Закрывает соединение с базой данных
 - b. Выполняет SQL-запрос
 - c. Создает курсор для работы с базой
 - d. Создает подключение к MySQL

Ответ: 1-d, 2-b, 3-a, 4-c



Выберите верные варианты ответа

Выберите корректные утверждения или несколько:

1. `cursor.execute()` можно вызывать до создания соединения
2. Курсор автоматически закрывается после каждого запроса
3. Один курсор можно использовать для нескольких запросов
4. `mysql.connector.connect()` возвращает объект курсора
5. `connection.open` проверяет статус соединения



Выберите верные варианты ответа

Выберите корректные утверждения или несколько:

- a. `cursor.execute()` можно вызывать до создания соединения
- b. Курсор автоматически закрывается после каждого запроса
- c. Один курсор можно использовать для нескольких запросов
- d. `pymysql.connect()` возвращает объект курсора
- e. `connection.open` проверяет статус соединения



Получение результатов запроса

Выполнение запросов



Пример

```
cursor.execute("SELECT * FROM employees")  
  
for row in cursor:  
    print(row)
```

Пояснения

- После выполнения запроса SELECT данные остаются в курсоре, пока не будут получены. Для их получения курсор можно использовать **как итератор** в цикле
- Каждая итерация даёт **одну строку результата** (в виде кортежа)
- Все строки читаются **последовательно**, до окончания выборки



Параметризованные запросы

Важно



При выполнении запросов в базу данных **никогда нельзя подставлять значения напрямую в строку SQL**. Это может привести к **SQL-инъекциям**.



SQL-инъекция

Это тип уязвимости, при котором злоумышленник может вставить произвольный SQL-код в запрос, чтобы получить несанкционированный доступ к данным, изменить или удалить информацию в базе данных

Пример

```
# Прямое включение пользовательского ввода
user_input = "1 OR 1=1"
sql = f"SELECT * FROM employees WHERE employee_id = {user_input}"
cursor.execute(sql) # выполнит: SELECT * FROM employees WHERE employee_id = 1 OR 1=1
# Получены все записи, вместо одной
for r in cursor:
    print(r)
```

Использование параметризованных запросов

```
cursor.execute(
    "SELECT * FROM table_name WHERE column1 = %s AND column2 > %s",
    (value1, value2)
)
```

- Первый аргумент — SQL-запрос с плейсхолдерами %s
- Второй аргумент — **последовательность значений** (кортеж или список), которые подставляются вместо %s

Пример: несколько значений

```
cursor.execute(
    "SELECT * FROM employees WHERE department_id = %s OR salary > %s",
    (60, 20000)
)
for r in cursor:
    print(r)
```

- Подставляются оба значения в соответствующие %s.
- Порядок значений в последовательности должен совпадать с порядком %s в запросе.

Пример: одно значение

```
cursor.execute(
    "SELECT * FROM employees WHERE department_id = %s",
    (100,) # Обязательно запятая!
)
for r in cursor:
    print(r)
```

- Если передаётся только одно значение, нужно ставить запятую после него, чтобы создать кортеж.

Преимущества параметризованных запросов



Безопасность — защита от SQL-инъекций



Универсальность — одно и то же выражение можно выполнять с разными данными



Автоматическое экранирование — библиотека сама правильно обрабатывает типы данных (строки, числа и т.д.)



Улучшение производительности — сервер базы данных может кэшировать план выполнения запроса

Именованные параметры



Вместо позиционных %s, в запросах можно использовать **именованные параметры**. Это делает код **более читабельным**, особенно когда передаётся много значений.

Именованные параметры

```
cursor.execute(
    "SELECT * FROM table_name WHERE column1 = %(param1)s AND column2 > %(param2)s",
    {"param1": value1, "param2": value2}
)
```

- В SQL-запросе используются плейсхолдеры вида %(name)s
- Вторым аргументом передаётся словарь, где ключи соответствуют плейсхолдерам
- Порядок ключей в словаре не важен — главное, чтобы ключи словаря совпадали с именами плейсхолдеров в запросе

Пример

```

cursor.execute(
    "SELECT * FROM employees WHERE department_id = %(dep_id)s OR salary > %(min_salary)s",
    {"min_salary": 20000, "dep_id": 60}
)
for r in cursor:
    print(r)

```

Преимущества



Повышается читаемость запроса



Удобно, когда значения приходят как словарь

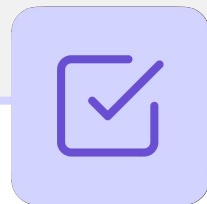


Легче избежать ошибок с порядком аргументов



Контекстный менеджер with

Важно



Чтобы упростить работу с соединением и курсором, можно использовать конструкцию `with`, которая **автоматически закрывает** ресурсы даже при ошибках.

Это делает код **чище** и **защищает от утечек соединений**.

Пример

```
with pymysql.connect(
    host='ich-db.edu.itcareerhub.de',
    user='ich1',
    password='password',
    database='hr'
) as connection: # автоматически закрывает connection
    with connection.cursor() as cursor: # автоматически закрывает cursor
        cursor.execute("SELECT * FROM employees")
        for row in cursor:
            print(row)
```

Особенности



Блок `with` автоматически вызывает `close()` для курсора и/или соединения



Код становится компактнее и безопаснее



Даже если внутри блока произойдёт ошибка — ресурсы **корректно закроются**



ВОПРОСЫ





ЗАДАНИЯ





Выберите верные варианты ответа

Зачем использовать параметризованные запросы?

- a. Для сокращения кода
- b. Для защиты от SQL-инъекций
- c. Для правильной подстановки значений и экранирования
- d. Для красивого синтаксиса



Выберите верные варианты ответа

Зачем использовать параметризованные запросы?

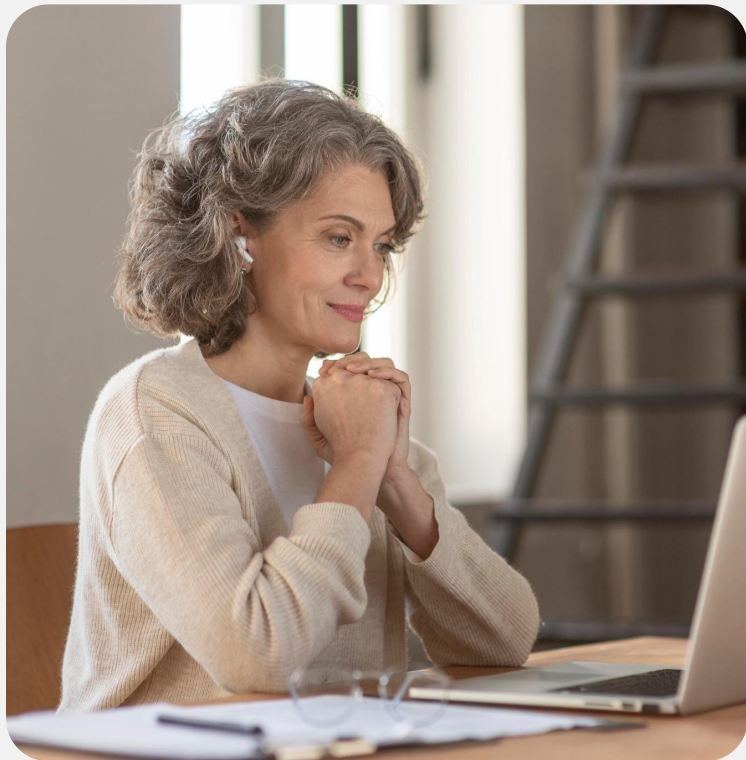
- a. Для сокращения кода
- b. Для защиты от SQL-инъекций
- c. Для правильной подстановки значений и экранирования
- d. Для красивого синтаксиса



Выберите верные варианты ответа

Какие утверждения верны для конструкции с **with**?

- a. Соединение автоматически закрывается после блока `with`
- b. Внутри него можно использовать `with connection.cursor()`
- c. Он помогает автоматически закрыть соединение даже при ошибке
- d. Его можно использовать только для `SELECT`



Выберите верные варианты ответа

Какие утверждения верны для конструкции с **with?**

- a. Соединение автоматически закрывается после блока **with**
- b. Внутри него можно использовать **with connection.cursor()**
- c. Он помогает автоматически закрыть соединение даже при ошибке
- d. Его можно использовать только для SELECT



Метод **commit**

Метод commit



Пояснения

Чтобы окончательно записать изменения в базу данных, необходимо вызвать метод `commit()`

Синтаксис

```
connection.commit()
```

Причины вызова commit



Без `commit()` изменения **не сохраняются** и будут **потеряны** при закрытии соединения



Это поведение обеспечивает **безопасность**: изменения применяются только после явного подтверждения



Создание базы данных и таблиц (`CREATE DATABASE`, `CREATE TABLE`) **не требует вызова** `commit()` — эти команды сохраняются сразу.

Подключение к серверу с правами на изменения



Пояснения

- Чтобы создавать базы и таблицы, подключитесь к серверу с правами на изменения
- Обратите внимание, что базу данных при подключении можно не указывать и выбрать ее позже

Код

```
config = {'host': 'ich-edit.edu.itcareerhub.de',
          'user': 'ich1',
          'password': 'ich1_password_ilovedbs',
          }

connection = pymysql.connect(**config)
```

Пример: создание базы и таблицы sales

```
with connection.cursor() as cursor:
    # Создание новой базы данных (если ещё не существует)
    cursor.execute("CREATE DATABASE IF NOT EXISTS market")
    cursor.execute("USE market")

    # Создание таблицы продаж
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS sales (
            id INT AUTO_INCREMENT PRIMARY KEY,
            item_name VARCHAR(100),
            quantity INT,
            price DECIMAL(10, 2),
            sale_date DATE
        )
    """)
```

Пример: вставка данных

```
with connection.cursor() as cursor:  
    cursor.execute(  
        "INSERT INTO sales (item_name, quantity, price, sale_date) VALUES (%s, %s, %s,  
        %s)",  
        ("Keyboard", 2, 45.50, "2024-06-15")  
    )  
    connection.commit() # Сохраняем изменения
```

Пример: обновление данных

```
with connection.cursor() as cursor:  
    cursor.execute(  
        "UPDATE sales SET quantity = %s WHERE item_name = %s",  
        (3, "Keyboard")  
    )  
    connection.commit()
```

Пример: удаление данных

```
with connection.cursor() as cursor:  
    cursor.execute(  
        "DELETE FROM sales WHERE item_name = %s",  
        ("Keyboard",)  
    )  
    connection.commit()
```

Метод executemany



Метод `executemany()` используется для **многократного выполнения одного и того же SQL-запроса** с разными данными. Это удобно и эффективно при массовом добавлении, обновлении или удалении строк.

Пример

```
with connection.cursor() as cursor:
    cursor.executemany(
        "INSERT INTO sales (item_name, quantity, price, sale_date) VALUES (%s, %s, %s, %s)",
        [
            ("Notebook", 3, 19.99, "2024-06-15"),
            ("Pen", 10, 1.99, "2024-06-16"),
            ("Bag", 1, 49.90, "2024-06-17")
        ]
    )
    connection.commit()
```

Особенности



Работает только с **изменяющими запросами** (INSERT, UPDATE, DELETE)



Вторым аргументом передаётся **список кортежей**



Экономит ресурсы, т.к. запрос компилируется один раз, а выполняется много



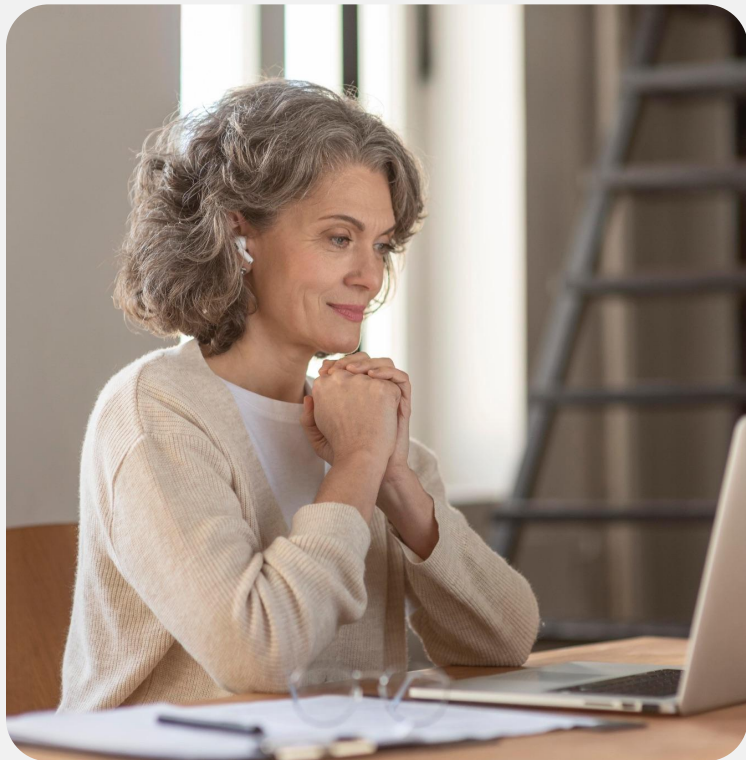
ВОПРОСЫ





ЗАДАНИЯ





Выберите верные варианты ответа

После какого действия нужно вызывать
`commit()`?

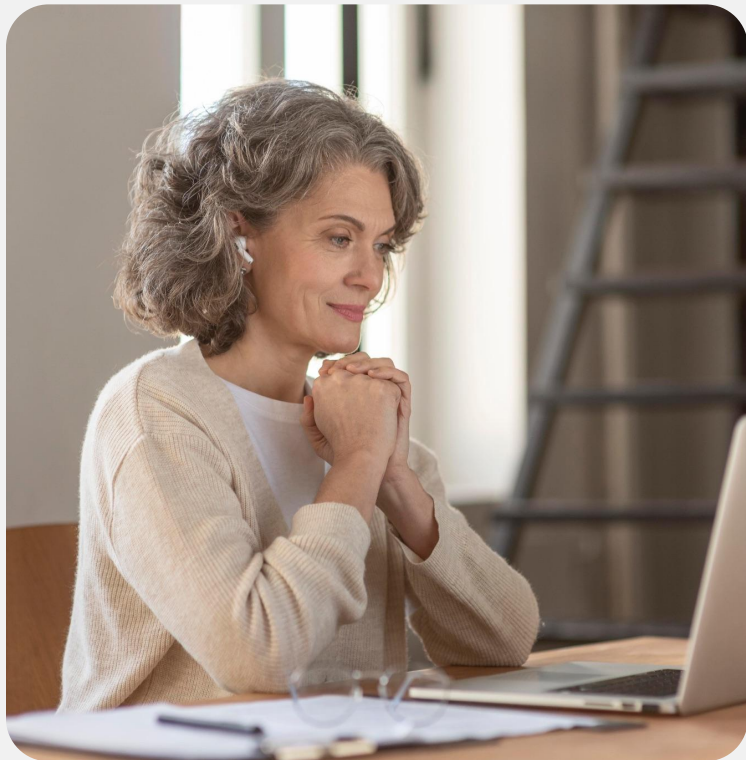
- a. SELECT
- b. INSERT
- c. UPDATE
- d. DELETE



Выберите верные варианты ответа

После какого действия нужно вызывать
commit()?

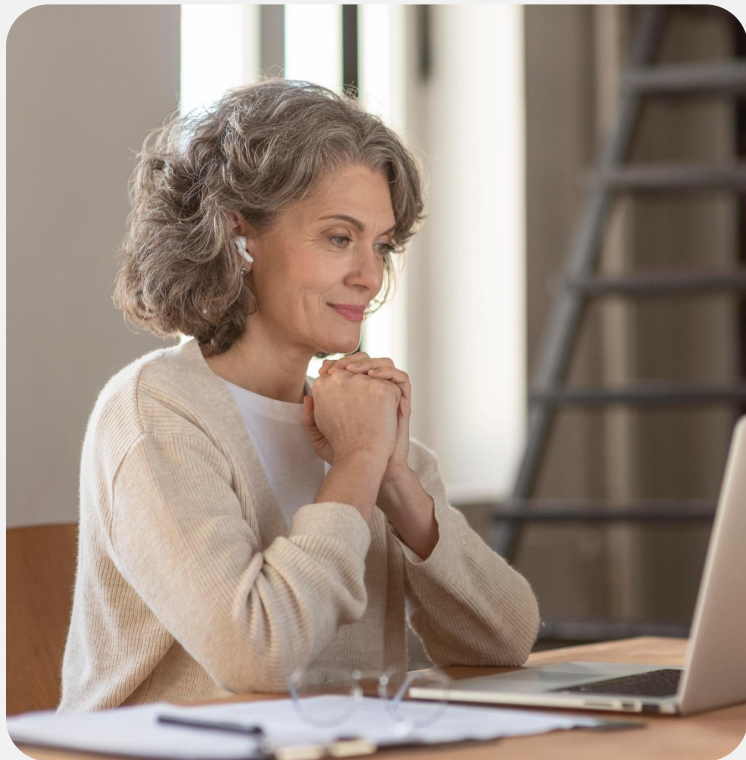
- a. SELECT
- b. INSERT
- c. UPDATE
- d. DELETE



Выберите верный вариант ответа

Что делает метод `executemany()`?

- a. Выполняет SQL-запрос указанное количество раз
- b. Выполняет SQL-запрос указанное количество раз или до первого успешного выполнения
- c. Позволяет выполнять один SQL-запрос с разными данными



Выберите верный вариант ответа

Что делает метод `executemany()`?

- a. Выполняет SQL-запрос указанное количество раз
- b. Выполняет SQL-запрос указанное количество раз или до первого успешного выполнения
- c. Позволяет выполнять один SQL-запрос с разными данными



Выберите верный вариант ответа

Что произойдёт, если после INSERT не вызвать commit()?

- a. Запрос не выполнится
- b. Изменения будут сохранены автоматически
- c. Изменения будут потеряны после закрытия соединения



Выберите верный вариант ответа

Что произойдёт, если после INSERT не вызвать commit()?

- a. Запрос не выполнится
- b. Изменения будут сохранены автоматически
- c. Изменения будут потеряны после закрытия соединения



ПРАКТИЧЕСКАЯ РАБОТА



1. Список всех департаментов

Используя базу данных `hr`, выведите названия всех департаментов из таблицы `departments`. Каждое название должно отображаться с новой строки и иметь номер.

Пример вывода:

```
1. Administration
2. Marketing
3. Purchasing
...
27. Payroll
```

2. Сотрудники выбранного департамента

Добавьте к предыдущей программе возможность выбора департамента. Пусть пользователь введёт название из списка. Далее выведите всех сотрудников этого департамента, указав имя, фамилию и должность.

Пример вывода:

```
Enter department: Marketing
Michael Hartstein – Marketing Manager
Pat Fay – Marketing Representative
...
```



ДОМАШНЕЕ ЗАДАНИЕ



Домашнее задание

1. Список всех стран

Используя базу данных world, выведи **названия всех стран** из таблицы country. Каждое название должно отображаться с новой строки и иметь номер.

Пример вывода:

```
1. Aruba
2. Afghanistan
3. Angola
...
239. Zimbabwe
```

Домашнее задание

2. Города выбранной страны

Добавьте к предыдущей программе возможность выбора страны. Пользователь должен ввести название страны. Далее выведите **все города этой страны и их численность населения**.

Пример вывода:

Введите страну: Germany

Berlin – 3386667

Hamburg – 1704735

Munich [München] – 1194560

...

Заключение

